

slopware nedir, nasıl fark edilir, ne yapmalı?

çalışıyor gibi görünen bir şeyle gerçekten ayakta duran bir şey arasındaki mesafeyi ölçmeyi öğreten bir müfredat. sıfırdan kurucu seviyesine.

2 / 25 bölüm · pdf · yayındaki bütün bölümler tek sayfada yazdırmak ya da pdf için.

01 dear connections (intro)

02 guys, I am on
localhost

01 dear connections (intro)

Herkes bu kelimeyi söylüyor ama kimse tanımını bilmiyor.

Bir şeyi ekrana getirmek hiç bu kadar ucuz olmamıştı. Bir cümle yazıyorsun, karşına çalışan bir arayüz çıkıyor, tıklıyorsun tepki veriyor, rengi de güzel. On dakika önce ortada hiçbir şey yokken şimdi elinde bir "ürün" var, ve o ürün sana kendini gerçek gibi hissettiriyor. İşte bu his yüzünden bu seriyi yazıyorum, çünkü çalışıyor gibi görünen bir şeyle gerçekten ayakta duran bir şey arasındaki mesafeyi artık kimse ölçmüyor.

Slopware, o mesafenin içine düşen yazılıma verdiğim ad. Demosu var, ekranı var, belki bir de landing sayfası var. Ama açıp kapattığında hafızası siliniyor, ya da sırlarını herkesin görebileceği bir yere koymuş, ya da testleri hiç koşmadan kendini yeşil sanıyor. Kötü niyetli bir şey değil bu, çoğu zaman heyecanlı bir insanın hızla yaptığı bir şey, ve o insanın kimse ona doğrusunu göstermemiş olması tek suçu.

Hemen bir yanlış anlaşılmayı kapatayım. Slopware demek "yapay zeka yazdı demek" değil. Ben de modele kod yazdırıyorum, bu seriyi okurken de yazdıracağım, ve projelerimin içinde açıkça wrapper olan şeyler var, onları saklamıyorum. Mesele kodu kimin yazdığı değil, o kodun arkasında bir sistem olup olmadığı. Bir şeyin tekrar tekrar, sen bakmıyorken, senin bilmediğin girdilerle doğru çalışacağına dair bir sebebin var mı, işte bütün soru bu.

Neden şimdi diye soruyorsan, cevap üretimin fiyatında. Token ucuzladı, üretim hızlandı, ve bir fikirle çalışan bir

demo arasındaki mesafe neredeyse sıfıra indi. Bu harika bir şey, benim gibi on bir yaşından beri kod yazan biri için bile harika. Ama aynı ucuzluk şunu da yaptı, artık piyasada birbirine benzeyen, hepsi ilk beş saniyede etkileyici duran, altıncı saniyede dağılan çok fazla şey var. İnsanlar bunu görüyor ve "ai slop" deyip geçiyor, ama geçmek yetmiyor, çünkü o sözü söyleyen kişi kendi projesinin de aynı çukurda olup olmadığını bilmiyor.

Bu seri tam olarak onu öğretecek. Her bölümde bir işaret alacağız, o işaretin neden ortaya çıktığını anlatacağım, kendi kodunda nasıl fark edeceğini göstereceğim, ve sonra doğrusunun nasıl kurulduğunu kendi projelerimden göstereceğim. Çünkü ben bu hataların hepsini yaptım, ve bazılarını düzeltirken kurduğum şeyler şu anda çalışıyor. stitchu'da bir kalıbın dikilebilir olup olmadığını kontrol eden bir denetim var, lulumelon'un mühürlü bir kaydı var, rabadon bir komutun gerçekten başarılı bitip bitmediğine karar veriyor. Bunların hepsi bir bölümde "işte doğrusu böyle görünüyor" diye önüne gelecek.

Şimdilik altı bölüm var:

localhost:3000 cesareti. Deploy edilmemiş bir şeyin ürün sanılması. Kendi makinende çalışan bir demoyla, senin uyurken başkasının kullandığı bir sistem arasındaki fark.

Hafızasız sistemler. Her yenilemede seni unutan uygulama. Veri nerede durur, state ne demektir, ve neden bir şeyin gerçekten var olması için bir yere yazılması gerekir.

Sır tutamayanlar. env dosyası nedir, API anahtarını tarayıcıya gömünce ne olur, ve bir sabah uyandığında faturayı neden senin ödediğini.

Test etmeden yeşil sanmak. Suite'i olmayan yazılım, ve exit 0'ı başarı sanmak. Bilgisayarın sana "tamam" demesiyle işin doğru olması aynı şey değil, bunu tüketici diliyle anlatacağım.

Wrapper anatomisi. Bir wrapper ne zaman öğrenme aracıdır, ne zaman ürün taklidi olur. Bu bölümde kendi wrapper'larımı açacağım.

Havuzda bekleyen başlıklar da var, her şeyi koda gömmek, hata mesajını try/catch içinde sessizce yutmak, ve "yapay zeka yazdı ben okumadım" kodu. Seri tutarsa onlarla dokuza çıkarız.

Son bir şey. Burada kimseyi aşağılamıyoruz. Slopware yazan insan tembel değil, sadece kendisine kimsenin göstermediği bir şeyi bilmiyor, ve bu serinin varlık sebebi de o. Ben sana göstereceğim, sen de bir daha aynı çukura düşmeyeceksin, anlaşma bu kadar basit.

Haftaya localhost:3000 ile başlıyoruz.

02 guys, I am on localhost

Bu, demo ile ürün arasındaki ilk ve en keskin çizgi.

Bir şey yaptın, çalışıyor. Tarayıcında duruyor, tıklıyorsun tepki veriyor, rengi de güzel. Heyecanlanıp linki arkadaşına atıyorsun, ve o "açılmıyor" diyor. Sen tekrar deniyorsun, sende gayet açılıyor. Bir tuhafılık var ama ne olduğunu bilmiyorsun.

Olan şu: o link senin bilgisayarının kendi kendine verdiği bir adres. Dışarıdan kimse oraya ulaşamaz, çünkü o adres dışarıda diye bir yer bilmiyor.

localhost tam olarak ne?

Bilgisayarında iki tür adres var. Biri dünyaya açılan kapı, diğeri **kendine yazdığın mektup**. localhost, yani 127.0.0.1, ikincisi. Oraya gönderdiğin her istek bilgisayarından hiç çıkmıyor, ağ kartına bile uğramıyor, aynı makinenin içinde dönüp sana geri geliyor. Buna loopback deniyor, ve adı tam olarak yaptığı şey: geri döngü.

Yani localhost:3000 yazdığında olan şey şu: "bu makinede, 3000 numaralı kapıda dinleyen program varsa onunla konuş". Arkadaşın aynı adresi yazdığında **kendi** makinesinde 3000 numaralı kapıya bakıyor, orada da hiçbir şey olmadığı için hata alıyor. Sana kızmıyor, sadece senin makineni göremiyor.

Peki 3000 ne? O bir port, yani kapı numarası. Bir bilgisayarda aynı anda onlarca program çalışabilir ve hepsi

ağdan konuşmak isteyebilir, o yüzden her biri kendine bir kapı seçiyor. Web sunucusu 3000'de, veritabanının 5432'de, başka bir şey 8080'de. Adresin sonundaki iki nokta ve rakam, "hangi kapı" demek.

Bunu gözünle gör

Anlatmakla olmuyor, çalıştırınca oluyor. Terminali aç, herhangi bir klasöre gir ve şunu yaz:

```
python3 -m http.server 8000
```

Şimdi tarayıcında localhost:8000 adresini aç. O klasördeki dosyaları göreceksin. Sunucun ayakta, çalışıyor, her şey yolunda görünüyor.

Şimdi telefonunu al ve aynı adresi telefonun tarayıcısına yaz. Açılmayacak. Wi-Fi'ın aynı bile olsa açılmayacak, çünkü telefonun localhost dediğinde **telefonun kendi kendisini** kastediyor.

Sunucuyu kapat ve bu sefer şöyle başlat:

```
python3 -m http.server 8000 --bind 0.0.0.0
```

0.0.0.0, "sadece kendi içimden değil, bütün ağ arayüzlerinden dinle" demek. Şimdi bilgisayarının yerel ağdaki adresini bul. Mac'te `ipconfig getifaddr en0`, Linux'ta `hostname -I`. Sonra telefonundan `http://192.168.1.x:8000` gibi bir şey yaz. Bu sefer açılacak.

Tek fark iki komut arasındaki o küçük ek. Ve az önce demo ile ürün arasındaki ilk çizgiyi kendi elinle çizdin.

Ama "telefonumdan açıldı" da ürün demek değil

Şimdi tehlikeli kısım geliyor, çünkü insanların çoğu burada duruyor ve "tamam işte, oldu" diyor. Olmadı. 0.0.0.0 ile açtığın şey sadece **aynı Wi-Fi ağındaki** cihazlara görünüyor. Kafedeki arkadaşın göremiyor, müşterin göremiyor, yatırımcın göremiyor.

Ve daha önemlisi: sen bilgisayarını kapattığın anda o şey ölüyor.

Gerçek bir ürünün senden bağımsız yaşaması gerekiyor. Sen uyurken, laptopun kapalıyken, sen tatildeyken de birinin girip kullanabilmesi gerekiyor. Bunun için o kodun senin makinende değil, **hep açık duran başka bir makinede** koşması lazım. Buna deploy diyoruz ve beşinci bölümün konusu bu.

Aradaki fark şuna benziyor: evinde pasta yapabiliyorsan bu güzel bir şey, ama pastane açmakla aynı şey değil. Pastanenin bir adresi var, sen orada olmasan da açık, ve sabah gelen müşteri kapıyı çalınca içeride birileri var.

Slopware'in en yaygın hali

Bu bölümün konusu teknik değil aslında, algı. İnsanlar localhost'ta çalışan bir şeyi görüp "ürünüm var" diyorlar, ve o cümleyi kurduktan sonra ilerlemeyi bırakıyorlar. Ekran görüntüsü alıyorlar, demo videosu çekiyorlar, hatta landing sayfası yapıyorlar, ama arkasında yayında hiçbir şey yok.

Ben bunu kendi projelerimde yaşadım, hem de bir kere değil. Şu anda makinemde iki tane bitmiş proje duruyor. Biri shortstorylong, içinde 2272 tane konu var. Diğer missingsemicolon, içinde 1206 tane gerçek mülakat sorusu var. İkisi de çalışıyor, ikisi de aylardır bitmiş durumda, ve **ikisi de canlıda değil**. Çünkü her seferinde "deploy sonra, önce şunu ekleyeyim" dedim.

O iki proje bitmiş ürün değil, benim makinemde duran iki klasör. Kimse kullanamıyor. Bitmiş olmalarının hiçbir anlamı yok, çünkü kimse göremiyor.



kontrol:

Adres çubuğundaki şey localhost ya da 127.0.0.1 ile mi başlıyor? Öyleyse o şey yayında değil, ne kadar iyi görünürse görünsün.

*Laptopunu kapat, telefonundan **mobil veriyle**, Wi-Fi kapalıyken aç:*

`https://senin-adresin`

Açılmıyorsa yayında değil. Ve bugüne kadar senden başka biri o adresi açtı mı? Bir kişi bile? Hayırsa elindeki şey bir ürün değil, bir prototip.

Kendi projende nasıl kontrol edersin?

Üç soru, üçü de yirmi saniye sürüyor.

Bir: Adres çubuğundaki şey localhost ya da 127.0.0.1 ile mi başlıyor? Öyleyse o şey henüz yayında değil, ne kadar iyi

görünürse görünsün.

İki: Laptopunu kapatınca ölüyor mu? Kapat ve telefonundan **mobil veriyle**, Wi-Fi kapalıyken aç. Açılmıyorsa yayında değil.

Üç: Bugüne kadar senden başka biri o adresi açtı mı? Bir kişi bile? Hayırsa, elindeki şey bir ürün değil, bir prototip. Prototip olması kötü bir şey değil, ama adını doğru koymak lazım, çünkü yanlış koyduğunda kendini yanlış yerde sanıyorsun.

Sonraki bölümde bir uygulamanın kaç parçadan oluştuğuna bakacağız, çünkü deploy'u anlamak için önce **neyi** deploy ettiğini bilmek gerekiyor.